SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q3 – Merging Two Merit Lists

Complete the Code (TODO 1)

```
int merge(const int a[], int na, const int b[], int nb, int out[])
{
   int i = 0, j = 0, k = 0;
   while (i < na && j < nb) {
      if (a[i] >= b[j]) out[k++] = a[i++];   /* >= : on a tie campus A comes first */
      else              out[k++] = b[j++];
   }
   while (i < na) out[k++] = a[i++];          /* rest of A, if any */
   while (j < nb) out[k++] = b[j++];          /* rest of B, if any */
   return k;
}
```

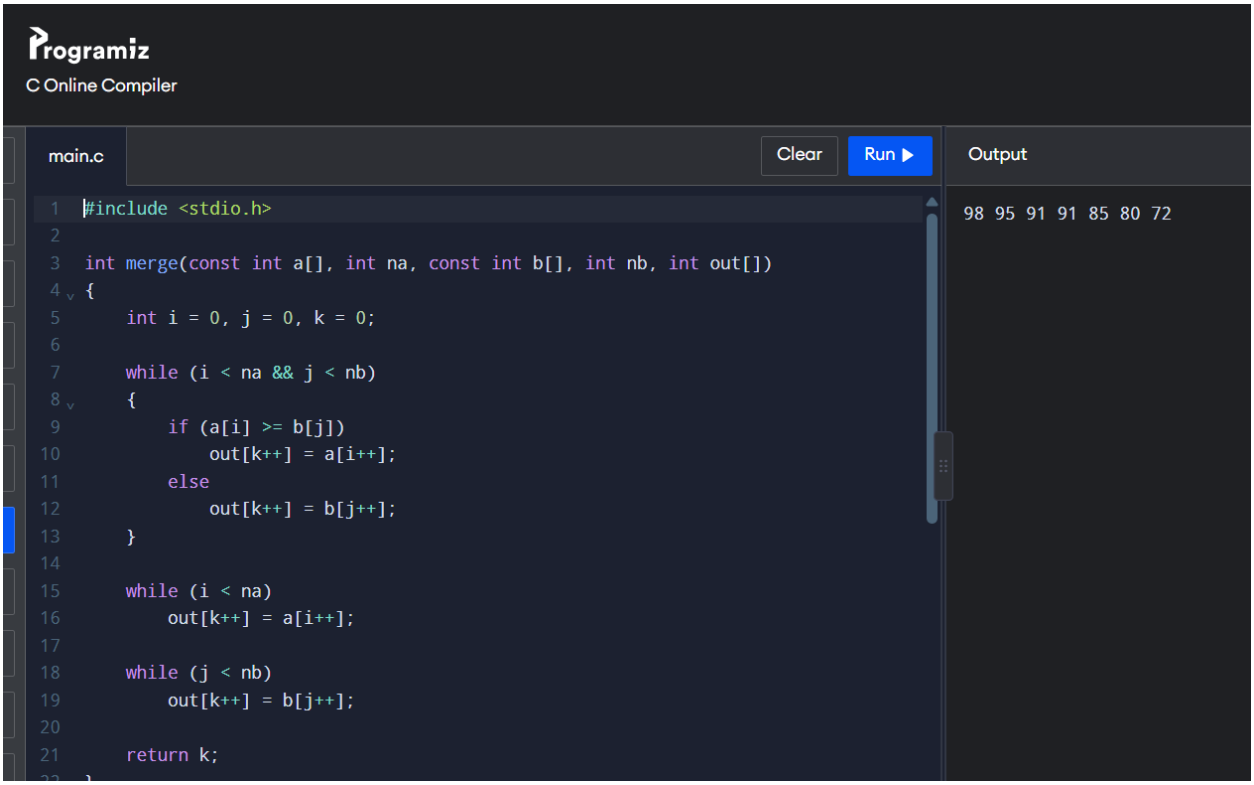
Output (matches expected):

```
98 95 91 91 85 80 72
```

Tests:

```
B empty : 98 91 85 72        (0 comparisons)
A empty : 95 91 80           (0 comparisons)
all equal (50 50 50 + 50 50): 50 50 50 50 50   (3 comparisons)
```

With an empty list the first while condition is false immediately, so no comparison is made and the matching copy loop transfers the whole other list. The merged lists were also checked to be in descending order for random 1,000 and 10,000 element lists.



Student – Analysis and Comparison

Comparisons: each comparison writes exactly one element to out[], and the last element is copied without a comparison, so the worst case is m + n − 1 (the lists interleave); the best case is min(m, n) (every element of the shorter list is larger than all of the other). Extra memory: the output array of m + n ints.

m = n	Merge comparisons (measured)	Concatenate + qsort comparisons (measured)	Extra memory (merge)
1,000	1,994 (max 1,999)	11,858	2,000 ints
10,000	19,999 (max 19,999)	149,215	20,000 ints

Merging does about 6 to 7.5 times fewer comparisons than concatenating and sorting again, and the gap grows with size, because merging uses the fact that both lists are already ordered. Tests also covered one empty list (0 comparisons) and all scores equal (the result is still correct and in order).

Student – Merge Design and Correctness

```
MERGE(a, na, b, nb, out):
  i = 0 ; j = 0 ; k = 0
  while i < na and j < nb:
    if a[i] >= b[j]: out[k] = a[i] ; i = i + 1
    else:          out[k] = b[j] ; j = j + 1
    k = k + 1
  while i < na: out[k] = a[i] ; i = i + 1 ; k = k + 1
  while j < nb: out[k] = b[j] ; j = j + 1 ; k = k + 1
  return k
```

Why the work is linear: every comparison moves one element permanently into out[] and never touches it again, so there are at most m + n − 1 comparisons and every element is written once. (Strictly, the element that loses a comparison stays and may be compared again with the next element of the other list; what is true is that no element is compared after it has been output.)

Copying the remainder: when one list ends, everything left in the other list is already in descending order and is smaller than or equal to everything already output, so it can be copied directly, with no more comparisons.

Equal scores: using >= takes from campus A first on a tie, so equal scores keep the order campus A, then campus B (the merge is stable). With > the order would flip to B first. The test 91 (A) and 91 (B) gives A's 91 first. The rule should be chosen by the admissions office, and the comparison operator is the only line to change.

Joint Engineering Decision – merging in place

Prefer an in-place merge when memory is the constraint (very large lists, a small device) or when the first array already has spare room at its end. Cost: a simple in-place merge shifts elements for every insertion: measured 499,500 shifts for two interleaved lists of 1,000 (about m·n/2), against 1,994 comparisons and 2,000 copies for the merge with a third array. If the first array has room for m + n elements, merging from the back (largest first, writing from the end) is O(m + n) with no third array and no shifting. True in-place algorithms with no spare space and O((m + n) log) time exist but are complicated, so the cost is either time (O(m·n)) or code complexity.

Alternative Concept and Complexity Analysis

Approach	Time	Extra space	Comment
Merge into third array	O(m + n)	O(m + n)	1,994 comparisons for m = n = 1,000
Copy into one array + qsort()	O((m + n) log(m + n))	O(1) to O(log n) (qsort stack) – the combined array is needed anyway	11,858 comparisons; ignores existing order
Simple in-place merge (shifting)	O(m · n)	O(1)	499,500 shifts for m = n = 1,000